Assignment 9

Chul Hee Kim

2021 11 12

Problem 1

a-)

```
data("uswages")
head(uswages)
```

```
##         wage educ exper race smsa ne mw so we pt
## 6085  771.60   18    18    0    1  1  0  0  0  0
## 23701 617.28   15    20    0    1  0  0  0  1  0
## 16208 957.83   16     9    0    1  0  0  1  0  0
## 2720  617.28   12    24    0    1  1  0  0  0  0
## 9723  902.18   14    12    0    1  0  1  0  0  0
## 22239 299.15   12    33    0    1  0  0  0  1  0
```

```
str(uswages)
```

```
## 'data.frame':    2000 obs. of  10 variables:
##  $ wage : num  772 617 958 617 902 ...
##  $ educ : int  18 15 16 12 14 12 16 16 12 12 ...
##  $ exper: int  18 20 9 24 12 33 42 0 36 37 ...
##  $ race : int  0 0 0 0 0 0 0 0 0 0 ...
##  $ smsa : int  1 1 1 1 1 1 1 1 1 0 ...
##  $ ne   : int  1 0 0 1 0 0 0 0 0 0 ...
##  $ mw   : int  0 0 0 0 1 0 0 1 0 1 ...
##  $ so   : int  0 0 1 0 0 0 1 0 0 0 ...
##  $ we   : int  0 1 0 0 0 1 0 0 1 0 ...
##  $ pt   : int  0 0 0 0 0 0 1 1 1 0 ...
```

```
uswages.sub <- uswages[,6:9]
table(rowSums(uswages.sub)==1)
```

```
##
## TRUE
## 2000
```

There are four categorical variables related to region, and we want to check if all of them are necessary. The result above tells us that the sum of every row of *ne, mw, so, we* predictors is 1, which means that one of four variables should be removed. We will remove *we* from *uswages* dataset.

```
uswages <- uswages[,-9] # remove 'we' variable

f1 <- lm(wage ~ ., data=uswages)
summary(f1)
```

```
##
```

```
## Call:
## lm(formula = wage ~ ., data = uswages)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -875.7 -213.8  -53.3  128.5 7505.5
##
## Coefficients:
##              Estimate Std. Error t value    Pr(>|t|)
## (Intercept) -203.9184    53.6126  -3.804    0.000147 ***
## educ          48.8034     3.2489  15.022     < 2e-16 ***
## exper          9.1353     0.7262  12.579     < 2e-16 ***
## race        -119.1585    35.1922  -3.386    0.000723 ***
## smsa         115.6783    21.7386   5.321 0.000000115 ***
## ne           -53.9265    27.9738  -1.928    0.054028 .
## mw           -60.1990    27.3487  -2.201    0.027839 *
## so           -50.4333    26.3703  -1.913    0.055955 .
## pt          -336.2156    31.9381 -10.527     < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 412.1 on 1991 degrees of freedom
## Multiple R-squared:  0.2001, Adjusted R-squared:  0.1969
## F-statistic: 62.25 on 8 and 1991 DF,  p-value: < 2.2e-16
```

The initial regression model including all predictors shows insignificant variables if α=0.05, but it can be argued since their p-values are very close to 0.05. R^2 is 0.2, which is not very good.

```r
ols_step_both_p(f1, prem=0.05)
```

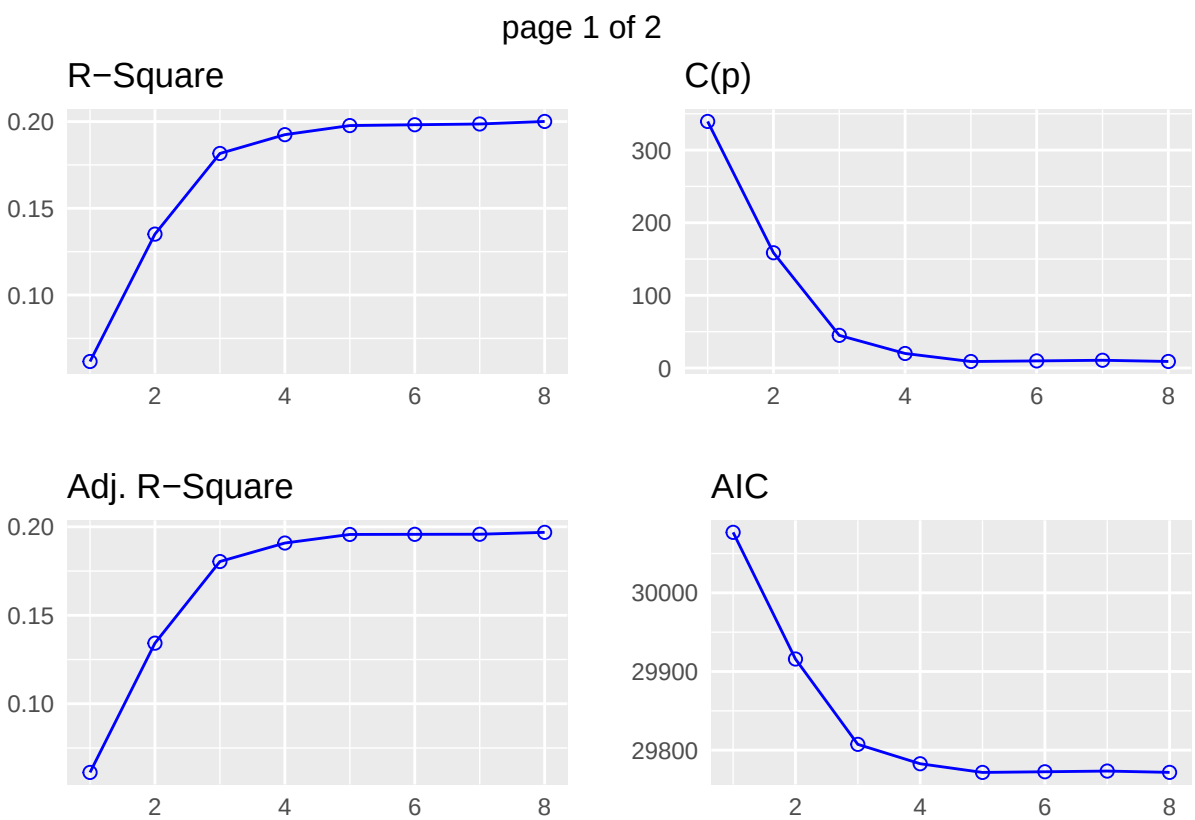
```
##
##                                  Stepwise Selection Summary
## -----------------------------------------------------------------------------------------
##                       Added/                    Adj.
## Step    Variable     Removed     R-Square    R-Square      C(p)          AIC          RMSE
## -----------------------------------------------------------------------------------------
##    1       educ      addition       0.062       0.061    339.4730    30076.8956    445.5394
##    2      exper      addition       0.135       0.134    158.6640    29915.8785    427.8538
##    3         pt      addition       0.182       0.180     44.9450    29807.3688    416.2994
##    4       smsa      addition       0.192       0.191     19.9850    29782.7211    413.6389
##    5       race      addition       0.198       0.196      8.9290    29771.6878    412.3967
## -----------------------------------------------------------------------------------------
```

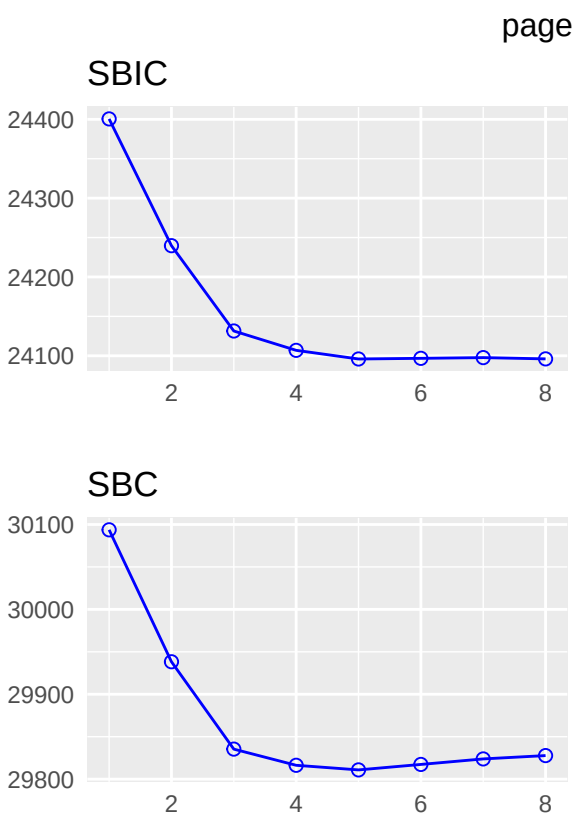
```r
k1 <- ols_step_best_subset(f1)
k1
```

```
##             Best Subsets Regression
## ----------------------------------------------
## Model Index    Predictors
## ----------------------------------------------
##      1         educ
##      2         educ exper
##      3         educ exper pt
##      4         educ exper smsa pt
##      5         educ exper race smsa pt
##      6         educ exper race smsa mw pt
```

```
##      7         educ exper race smsa ne mw pt
##      8         educ exper race smsa ne mw so pt
## ----------------------------------------------
##
##                                                                    Subsets Regression Summary
## ---------------------------------------------------------------------------------------------------
##                         Adj.        Pred
## Model    R-Square    R-Square    R-Square      C(p)         AIC           SBIC          SBC
## ---------------------------------------------------------------------------------------------------
##   1        0.0617      0.0612      0.0572    339.4734    30076.8956    24400.5256    30093.6983    3
##   2        0.1351      0.1343      0.1294    158.6637    29915.8785    24239.6888    29938.2821    3
##   3        0.1816      0.1804      0.1736     44.9451    29807.3688    24131.4689    29835.3733    3
##   4        0.1924      0.1908      0.1839     19.9851    29782.7211    24106.9172    29816.3265    3
##   5        0.1977      0.1957      0.1887      8.9294    29771.6878    24095.9521    29810.8941    3
##   6        0.1982      0.1957      0.1886      9.7564    29772.5127    24096.7883    29817.3199    3
##   7        0.1986      0.1958      0.1881     10.6577    29773.4113    24097.7000    29823.8194    3
##   8        0.2001      0.1969      0.1887      9.0000    29771.7405    24096.0677    29827.7495    3
## ---------------------------------------------------------------------------------------------------
## AIC: Akaike Information Criteria
##  SBIC: Sawa's Bayesian Information Criteria
##  SBC: Schwarz Bayesian Criteria
##  MSEP: Estimated error of prediction, assuming multivariate normality
##  FPE: Final Prediction Error
##  HSP: Hocking's Sp
##  APC: Amemiya Prediction Criteria
```

```
plot(k1)
```



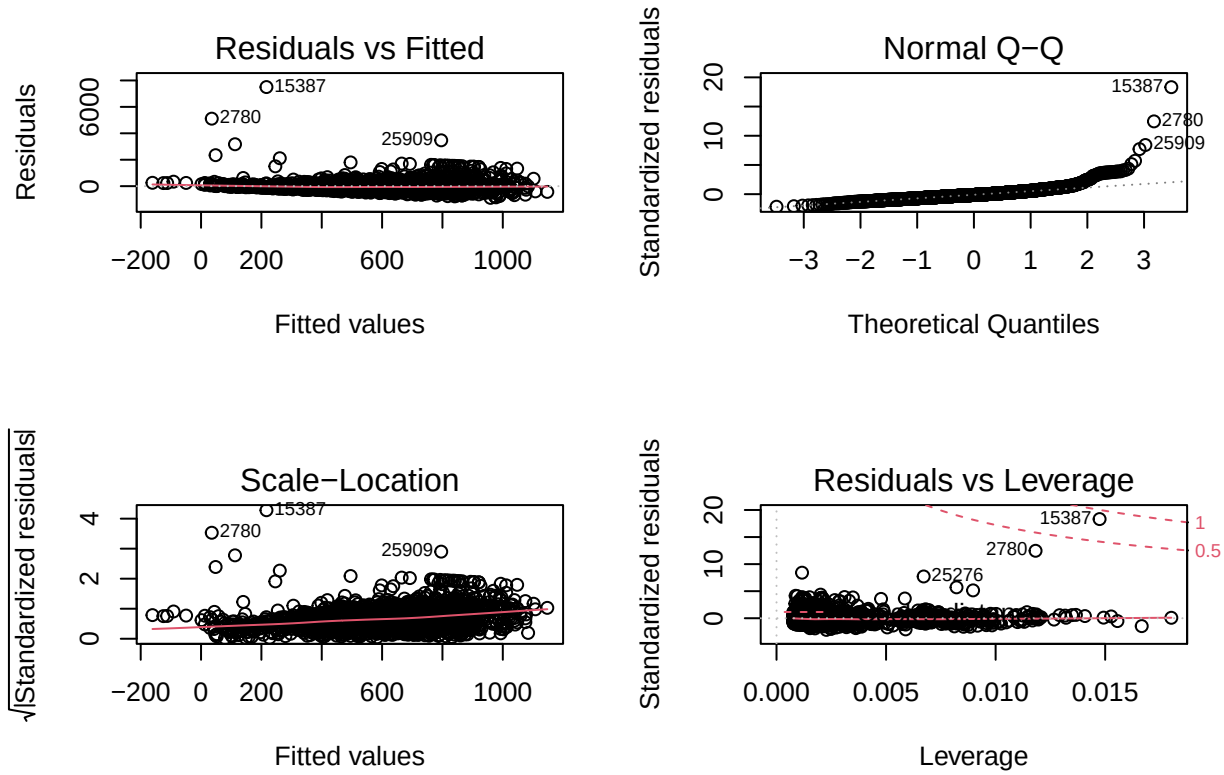


The stepwise regression selects five predictors, *educ, exper, pt, smsa, race*. The best subset regression also selects the same five predictors since Model 5 shows the best results given by adjusted R^2, C_p, and AIC. Now, let's check the regression model assumptions.

```
f1a <- lm(wage ~ educ+exper+pt+smsa+race, data=uswages)
summary(f1a)
```

```
##
## Call:
## lm(formula = wage ~ educ + exper + pt + smsa + race, data = uswages)
##
## Residuals:
##    Min     1Q Median     3Q    Max
## -885.8 -212.9  -56.8  128.9 7499.3
##
## Coefficients:
##              Estimate Std. Error t value     Pr(>|t|)
## (Intercept) -243.4879    50.8767  -4.786 0.0000018278 ***
## educ          48.6616     3.2478  14.983      < 2e-16 ***
## exper          9.0798     0.7259  12.509      < 2e-16 ***
## pt          -336.9503    31.9420 -10.549      < 2e-16 ***
## smsa         115.5466    21.5772   5.355 0.0000000954 ***
## race        -124.9292    34.6004  -3.611     0.000313 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 412.4 on 1994 degrees of freedom
## Multiple R-squared:  0.1977, Adjusted R-squared:  0.1957
## F-statistic: 98.26 on 5 and 1994 DF,  p-value: < 2.2e-16
```

```r
par(mfrow=c(2,2))
plot(f1a)
```




```r
vif(f1a)
```

```
##     educ    exper       pt     smsa     race
## 1.118935 1.107952 1.007190 1.009953 1.012483
```

```r
dwtest(f1a)
```

```
##
##  Durbin-Watson test
##
## data:  f1a
## DW = 1.8988, p-value = 0.01173
## alternative hypothesis: true autocorrelation is greater than 0
```

The residuals against fitted values plot shows a slight cone shape, and the scale-location plot shows $\sqrt{standardizedresiduals}$ increasing as the fitted values becomes larger. The QQ plot indicates that the error terms are approximately normally distributed but a little skewed right. All these plots indicate a few potential outliers.

VIFs show no multicollinearity issue.

Durbin-Watson test concludes that there is autocorrelation in the model.

b-)

```r
f1b <- lm(log(wage) ~ educ+exper+pt+smsa+race, data=uswages)
summary(f1b)
```

```
##
## Call:
```

```
## lm(formula = log(wage) ~ educ + exper + pt + smsa + race, data = uswages)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.5319 -0.3330  0.0495  0.3563  3.9435
##
## Coefficients:
##              Estimate Std. Error t value     Pr(>|t|)
## (Intercept)  4.725711   0.070545  66.989      < 2e-16 ***
## educ         0.086566   0.004503  19.223      < 2e-16 ***
## exper        0.016037   0.001006  15.934      < 2e-16 ***
## pt          -1.098583   0.044290 -24.804      < 2e-16 ***
## smsa         0.174543   0.029919   5.834 0.0000000063 ***
## race        -0.211327   0.047976  -4.405 0.0000111480 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5718 on 1994 degrees of freedom
## Multiple R-squared:  0.3843, Adjusted R-squared:  0.3827
## F-statistic: 248.9 on 5 and 1994 DF,  p-value: < 2.2e-16
```

```r
ols_step_both_p(f1b, prem=0.05)
```

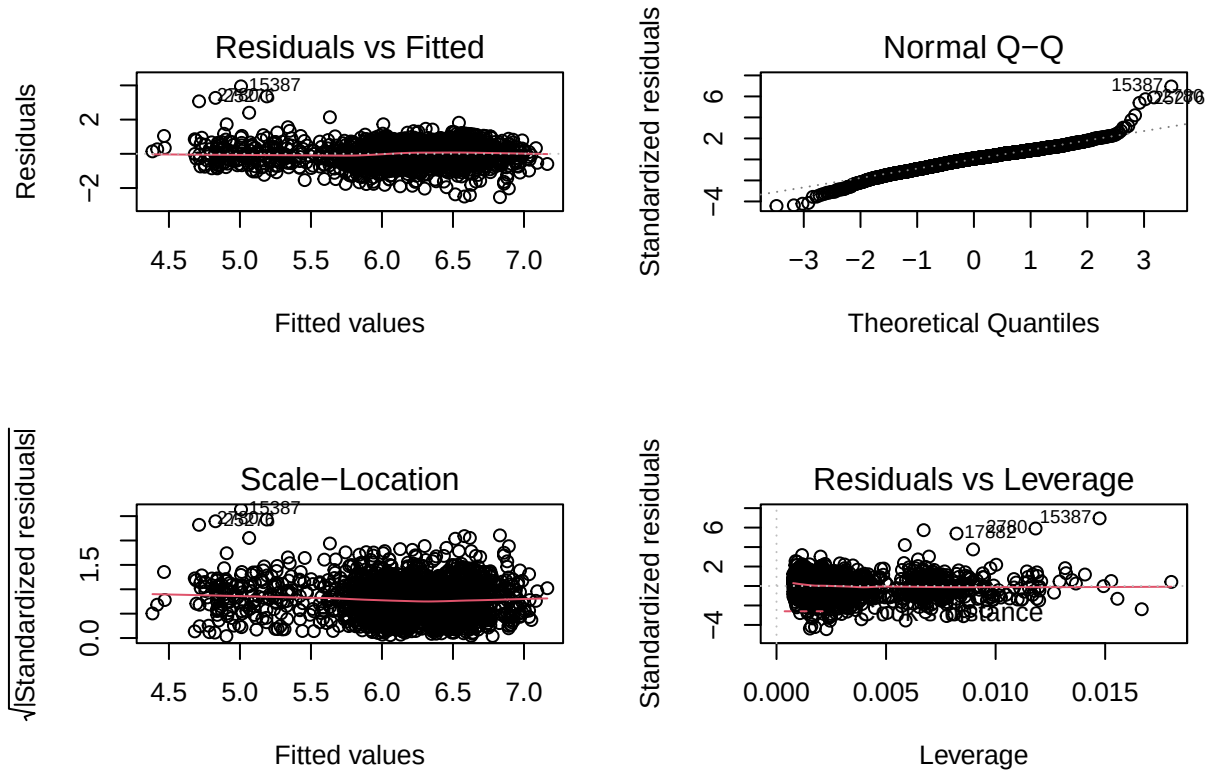
```
##
##                              Stepwise Selection Summary
## ---------------------------------------------------------------------------------------
##                      Added/                    Adj.
## Step    Variable    Removed     R-Square    R-Square      C(p)          AIC        RMSE
## ---------------------------------------------------------------------------------------
##    1       pt       addition       0.207       0.207    570.6000    3944.9102    0.6481
##    2      educ      addition       0.288       0.288    310.8810    3731.8044    0.6143
##    3     exper      addition       0.369       0.368     53.0990    3494.6388    0.5788
##    4      smsa      addition       0.378       0.377     23.4020    3465.3986    0.5745
##    5      race      addition       0.384       0.383      6.0000    3448.0318    0.5718
## ---------------------------------------------------------------------------------------
```

```r
f1b2 <- lm(log(wage) ~ educ+exper+pt+smsa+race, data=uswages)
summary(f1b2)
```

```
##
## Call:
## lm(formula = log(wage) ~ educ + exper + pt + smsa + race, data = uswages)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.5319 -0.3330  0.0495  0.3563  3.9435
##
## Coefficients:
##              Estimate Std. Error t value     Pr(>|t|)
## (Intercept)  4.725711   0.070545  66.989      < 2e-16 ***
## educ         0.086566   0.004503  19.223      < 2e-16 ***
## exper        0.016037   0.001006  15.934      < 2e-16 ***
## pt          -1.098583   0.044290 -24.804      < 2e-16 ***
## smsa         0.174543   0.029919   5.834 0.0000000063 ***
## race        -0.211327   0.047976  -4.405 0.0000111480 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.5718 on 1994 degrees of freedom
## Multiple R-squared:  0.3843, Adjusted R-squared:  0.3827
## F-statistic: 248.9 on 5 and 1994 DF,  p-value: < 2.2e-16
```

```
par(mfrow=c(2,2))
plot(f1b2)
```




```
vif(f1b2)
```

```
##     educ    exper       pt     smsa     race
## 1.118935 1.107952 1.007190 1.009953 1.012483
```

```
dwtest(f1b2)
```

```
##
##  Durbin-Watson test
##
## data:  f1b2
## DW = 2.0272, p-value = 0.7283
## alternative hypothesis: true autocorrelation is greater than 0
```

The residuals against fitted values plot no longer shows the cone shape, and the scale-location plot shows the red horizontal line, which is improved from the previous model. The QQ plot indicates that the error terms are approximately normally distributed but still a little skewed right. All these plots still indicate a few potential outliers.

VIFs show no multicollinearity issue.

Durbin-Watson test concludes that there is now no autocorrelation in the model.

R^2 is improved from 0.2 in part(a) to 0.38 in part(b), so all the results mentioned above indicate that the log-transformed model is the one we would choose over the one in part(a).

c-)

```
f1tree <- rpart(wage ~ ., data=uswages)
f1tree
```

```
## n= 2000
##
## node), split, n, deviance, yval
##       * denotes terminal node
##
##  1) root 2000 422680600.0  608.1179
##    2) educ< 15.5 1478 233173200.0  528.1470
##      4) exper< 11.5 529  40767740.0  375.7282 *
##      5) exper>=11.5 949 173265500.0  613.1097 *
##    3) educ>=15.5 522 153291700.0  834.5488
##      6) exper< 11.5 228  38122060.0  614.8371
##       12) pt>=0.5 36   1006887.0  240.6103 *
##       13) pt< 0.5 192  31128220.0  685.0047 *
##      7) exper>=11.5 294  95627870.0 1004.9370
##       14) pt>=0.5 14    715871.5  323.1150 *
##       15) pt< 0.5 280  88078240.0 1039.0290 *
```

```
pred.tree <- predict(f1tree, uswages)
summary(pred.tree)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   240.6   375.7   613.1   608.1   613.1  1039.0
```

```
MAE <- function(actual, predicted) {mean(abs(actual - predicted))}
SSE <- function(actual, predicted) {sum((actual - predicted)^2)}
R2 <- function(actual, predicted) {sum((actual - predicted)^2) /((length(actual)-1)*var(actual))}

mae.c <- MAE(uswages$wage, pred.tree)
sse.c <- SSE(uswages$wage, pred.tree)
r2.c <- 1-R2(uswages$wage, pred.tree)
pred.c <- c(mae.c, sse.c, r2.c)
```

d-)

```
f1tree.log <- rpart(log(wage) ~ ., data=uswages)
f1tree.log
```

```
## n= 2000
##
## node), split, n, deviance, yval
##       * denotes terminal node
##
##  1) root 2000 1058.91800 6.169782
##    2) pt>=0.5 185  116.46770 5.131671
##      4) exper< 2.5 72   18.98382 4.818452 *
##      5) exper>=2.5 113   85.91953 5.331244 *
##    3) pt< 0.5 1815  722.75870 6.275595
##      6) educ< 15.5 1343  466.39610 6.152972
##       12) exper< 11.5 448  149.09020 5.856391
```

```
##         24) exper< 2.5 75   28.63683 5.459724 *
##         25) exper>=2.5 373  106.27970 5.936149 *
##       13) exper>=11.5 895  258.17460 6.301428
##         26) educ< 9.5 142   50.85548 5.977715 *
##         27) educ>=9.5 753  189.63280 6.362474 *
##      7) educ>=15.5 472  178.71000 6.624500
##       14) exper< 11.5 192   65.25106 6.368627 *
##       15) exper>=11.5 280   92.26885 6.799955 *
```

```
pred.tree.log <- predict(f1tree.log, uswages)
summary(pred.tree.log)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   4.818   5.936   6.362   6.170   6.362   6.800
```

```
mae.d <- MAE(uswages$wage, exp(pred.tree.log))
sse.d <- SSE(uswages$wage, exp(pred.tree.log))
r2.d <- 1-R2(uswages$wage, exp(pred.tree.log))
pred.d <- c(mae.d, sse.d, r2.d)

tbl <- cbind(pred.c, pred.d)
colnames(tbl) <- c("Normal Tree", "Log-scaled Tree")
rownames(tbl) <- c("MAE", "SSE", "Rsquared")
kable(tbl, digits=3, align="c", caption="Comparison of Model Performance",
      booktabs=TRUE) %>% kable_styling(latex_options="HOLD_position")
```

Table 1: Comparison of Model Performance

	Normal Tree	Log-scaled Tree
MAE	262.672	242.109
SSE	334962426.509	341554926.459
Rsquared	0.208	0.192

The results are mixed as MAE is smaller for the log-scaled tree model, R^2 is higher for the normal tree model, but the differences are very small. There is no definite winner here, but given similar performances, we would choose the normal tree model because it is the simpler model and is easier to interpret.

e-)

```
set.seed(1023)
rf <- randomForest(wage ~ ., data = uswages)
rf
```

```
##
## Call:
##  randomForest(formula = wage ~ ., data = uswages)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 2
##
##           Mean of squared residuals: 163929.4
##                     % Var explained: 22.43
```

```
pred.rf <- predict(rf, uswages)
mae.e <- MAE(uswages$wage, pred.rf)
```

```r
sse.e <- SSE(uswages$wage, pred.rf)
r2.e <- 1-R2(uswages$wage, pred.rf)
pred.e <- c(mae.e, sse.e, r2.e)
```

f-)

```r
set.seed(1023)
bg <- ipred::bagging(wage ~ ., data = uswages, nbagg = 25)
pred.bg <- predict(bg, uswages)
mae.f <- MAE(uswages$wage, pred.bg)
sse.f <- SSE(uswages$wage, pred.bg)
r2.f <- 1-R2(uswages$wage, pred.bg)
pred.f <- c(mae.f, sse.f, r2.f)
```

g-)

```r
set.seed(1023)
bt <- gbm(wage ~ ., data = uswages)
```

```
## Distribution not specified, assuming gaussian ...
```

```r
pred.bt <- predict(bt, uswages)
```

```
## Using 100 trees...
```

```r
mae.g <- MAE(uswages$wage, pred.bt)
sse.g <- SSE(uswages$wage, pred.bt)
r2.g <- 1-R2(uswages$wage, pred.bt)
pred.g <- c(mae.g, sse.g, r2.g)

# Comparison of all models
tbl2 <- cbind(pred.c, pred.d, pred.e, pred.f, pred.g)
colnames(tbl2) <- c("Normal Tree", "Log-scaled Tree", "Random Forest", "Bagging", "Boosting")
rownames(tbl2) <- c("MAE", "SSE", "Rsquared")
kable(tbl2, digits=3, align="c", caption="Comparison of Model Performance",
      booktabs=TRUE) %>% kable_styling(latex_options="HOLD_position")
```

Table 2: Comparison of Model Performance

	Normal Tree	Log-scaled Tree	Random Forest	Bagging	Boosting
MAE	262.672	242.109	226.342	250.220	242.726
SSE	334962426.509	341554926.459	233511862.116	307594298.215	313372406.860
Rsquared	0.208	0.192	0.448	0.272	0.259

The table above suggests that the random forest performs significantly better than all other models.

Problem 2

a-)

```r
data("pima")
head(pima)
```

```
##   pregnant glucose diastolic triceps insulin  bmi diabetes age test
## 1        6     148        72      35       0 33.6    0.627  50    1
```

```
## 2        1      85        66      29       0 26.6    0.351  31    0
## 3        8     183        64       0       0 23.3    0.672  32    1
## 4        1      89        66      23      94 28.1    0.167  21    0
## 5        0     137        40      35     168 43.1    2.288  33    1
## 6        5     116        74       0       0 25.6    0.201  30    0
```

```
str(pima)
```

```
## 'data.frame':    768 obs. of  9 variables:
##  $ pregnant : int  6 1 8 1 0 5 3 10 2 8 ...
##  $ glucose  : int  148 85 183 89 137 116 78 115 197 125 ...
##  $ diastolic: int  72 66 64 66 40 74 50 0 70 96 ...
##  $ triceps  : int  35 29 0 23 35 0 32 0 45 0 ...
##  $ insulin  : int  0 0 0 94 168 0 88 0 543 0 ...
##  $ bmi      : num  33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
##  $ diabetes : num  0.627 0.351 0.672 0.167 2.288 ...
##  $ age      : int  50 31 32 21 33 30 26 29 53 54 ...
##  $ test     : int  1 0 1 0 1 0 1 0 1 1 ...
```

```
pima$test <- as.factor(pima$test)
set.seed(1023)
train.index <- createDataPartition(y = pima$test, p = 0.9, list = FALSE)
train.set <- pima[train.index,]
test.set <- pima[-train.index,]

f2t <- C5.0(train.set[,-9], train.set[,9])
f2t
```

```
##
## Call:
## C5.0.default(x = train.set[, -9], y = train.set[, 9])
##
## Classification Tree
## Number of samples: 692
## Number of predictors: 8
##
## Tree size: 18
##
## Non-standard options: attempt to group attributes
```

```
summary(f2t)
```

```
##
## Call:
## C5.0.default(x = train.set[, -9], y = train.set[, 9])
##
##
## C5.0 [Release 2.07 GPL Edition]      Wed Nov 17 06:02:04 2021
## -------------------------------
##
## Class specified by attribute `outcome'
##
## Read 692 cases (9 attributes) from undefined.data
##
## Decision tree:
##
```

```
## glucose > 127:
## :...bmi <= 29.9: 0 (68/22)
## :   bmi > 29.9:
## :   :...glucose > 157: 1 (79/10)
## :       glucose <= 157:
## :       :...diastolic > 90: 1 (9)
## :           diastolic <= 90:
## :           :...diastolic <= 61: 1 (14/1)
## :               diastolic > 61:
## :               :...age <= 30: 0 (34/11)
## :                   age > 30: 1 (48/16)
## glucose <= 127:
## :...bmi <= 26.4: 0 (116/2)
##     bmi > 26.4:
##     :...age > 28: 0 (158/63)
##         age <= 28:
##         :...triceps <= 0:
##             :...glucose <= 110: 0 (13)
##             :   glucose > 110:
##             :   :...diastolic > 72: 1 (5)
##             :       diastolic <= 72:
##             :       :...diastolic <= 52: 1 (2)
##             :           diastolic > 52: 0 (4)
##             triceps > 0:
##             :...triceps <= 29: 0 (77/2)
##                 triceps > 29:
##                 :...diabetes <= 0.463: 0 (35/2)
##                     diabetes > 0.463:
##                     :...diastolic > 68: 0 (13/1)
##                         diastolic <= 68:
##                         :...glucose <= 87: 0 (4)
##                             glucose > 87:
##                             :...glucose <= 117: 1 (9/1)
##                                 glucose > 117: 0 (4/1)
##
##
## Evaluation on training data (692 cases):
##
##      Decision Tree
##    ----------------
##    Size      Errors
##
##      18  132(19.1%)   <<
##
##
##     (a)   (b)    <-classified as
##    ----  ----
##     422    28    (a): class 0
##     104   138    (b): class 1
##
##
##  Attribute usage:
##
##  100.00% glucose
```

```
##  100.00% bmi
##   58.67% age
##   23.99% triceps
##   21.10% diastolic
##    9.39% diabetes
##
##
## Time: 0.0 secs
```

```
pred.f2t <- predict(f2t, train.set)
CrossTable(train.set$test, pred.f2t, prop.chisq = FALSE, prop.c = FALSE, prop.r = FALSE, dnn = c('actual
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  692
##
##
##                | predicted default
## actual default |         0 |         1 | Row Total |
## ---------------|-----------|-----------|-----------|
##              0 |       422 |        28 |       450 |
##                |     0.610 |     0.040 |           |
## ---------------|-----------|-----------|-----------|
##              1 |       104 |       138 |       242 |
##                |     0.150 |     0.199 |           |
## ---------------|-----------|-----------|-----------|
##   Column Total |       526 |       166 |       692 |
## ---------------|-----------|-----------|-----------|
##
##
```

b-)

```
f2.svm <- ksvm(test ~ ., data = train.set, kernel = "vanilladot")
```

```
##  Setting default kernel parameters
```

```
f2.svm2 <- ksvm(test ~ ., data = train.set, kernel = "rbfdot")

pred.svm <- predict(f2.svm, train.set)
pred.svm2 <- predict(f2.svm2, train.set)

CrossTable(train.set$test, pred.svm, prop.chisq = FALSE, prop.c = FALSE, prop.r = FALSE, dnn = c('actual
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |         N / Table Total |
```

```
## |-------------------------|
##
##
## Total Observations in Table:  692
##
##
##                | predicted default
## actual default |         0 |         1 | Row Total |
## ---------------|-----------|-----------|-----------|
##              0 |       395 |        55 |       450 |
##                |     0.571 |     0.079 |           |
## ---------------|-----------|-----------|-----------|
##              1 |       105 |       137 |       242 |
##                |     0.152 |     0.198 |           |
## ---------------|-----------|-----------|-----------|
##   Column Total |       500 |       192 |       692 |
## ---------------|-----------|-----------|-----------|
##
##
```

```
CrossTable(train.set$test, pred.svm2, prop.chisq = FALSE, prop.c = FALSE, prop.r = FALSE, dnn = c('actua
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  692
##
##
##                | predicted default
## actual default |         0 |         1 | Row Total |
## ---------------|-----------|-----------|-----------|
##              0 |       412 |        38 |       450 |
##                |     0.595 |     0.055 |           |
## ---------------|-----------|-----------|-----------|
##              1 |        91 |       151 |       242 |
##                |     0.132 |     0.218 |           |
## ---------------|-----------|-----------|-----------|
##   Column Total |       503 |       189 |       692 |
## ---------------|-----------|-----------|-----------|
##
##
```

We chose two of the most common kernels for our SVM, linear ("vanilladot") and Gaussian RBF kernel ("rbfdot"). The error rate is $160/692 = 23.12\%$ for the linear kernel and $129/692 = 18.64\%$ for the Gaussian RBF kernel. It seems that the latter performs better.

c-)

```
pred.f2t.test <- predict(f2t, test.set)
pred.svm.test <- predict(f2.svm, test.set)
```

```r
pred.svm2.test <- predict(f2.svm2, test.set)

CrossTable(test.set$test, pred.f2t.test, prop.chisq = FALSE, prop.c = FALSE, prop.r = FALSE, dnn = c('a
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  76
##
##
##                | predicted default
## actual default |         0 |         1 | Row Total |
## ---------------|-----------|-----------|-----------|
##              0 |        44 |         6 |        50 |
##                |     0.579 |     0.079 |           |
## ---------------|-----------|-----------|-----------|
##              1 |        10 |        16 |        26 |
##                |     0.132 |     0.211 |           |
## ---------------|-----------|-----------|-----------|
##   Column Total |        54 |        22 |        76 |
## ---------------|-----------|-----------|-----------|
##
##
```

```r
CrossTable(test.set$test, pred.svm.test, prop.chisq = FALSE, prop.c = FALSE, prop.r = FALSE, dnn = c('a
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  76
##
##
##                | predicted default
## actual default |         0 |         1 | Row Total |
## ---------------|-----------|-----------|-----------|
##              0 |        46 |         4 |        50 |
##                |     0.605 |     0.053 |           |
## ---------------|-----------|-----------|-----------|
##              1 |         8 |        18 |        26 |
##                |     0.105 |     0.237 |           |
## ---------------|-----------|-----------|-----------|
##   Column Total |        54 |        22 |        76 |
```

```
## ---------------|-----------|-----------|-----------|
##
##
```

```
CrossTable(test.set$test, pred.svm2.test, prop.chisq = FALSE, prop.c = FALSE, prop.r = FALSE, dnn = c('a
```

```
##
##
##    Cell Contents
## |-------------------------|
## |                       N |
## |         N / Table Total |
## |-------------------------|
##
##
## Total Observations in Table:  76
##
##
##                | predicted default
## actual default |         0 |         1 | Row Total |
## ---------------|-----------|-----------|-----------|
##              0 |        46 |         4 |        50 |
##                |     0.605 |     0.053 |           |
## ---------------|-----------|-----------|-----------|
##              1 |         7 |        19 |        26 |
##                |     0.092 |     0.250 |           |
## ---------------|-----------|-----------|-----------|
##   Column Total |        53 |        23 |        76 |
## ---------------|-----------|-----------|-----------|
##
##
```

The error rate is 16/76 = 21.05% for the tree model, 12/76 = 15.79% for SVM with the linear kernel, 11/76 = 14.47% for SVM with the Gaussian RBF kernel. The difference between the two SVM models is marginal, but the RBF kernel model also performed better with the training data, so we would select the SVM with the Gaussian RBF kernel as our final model.